Phase 1: Self-Supervised Pretraining

The model learns general wireless knowledge from unlabeled data by reconstructing masked inputs

Step 1

IQ Dataset
3,200 unlabeled samples
4-antenna MIMO recordings of various radio signals

Spectrogram Dataset
3,200 unlabeled samples
WiFi, LTE, Bluetooth, 5G-NR signal images

Step 2

Random Masking — 70% of tokens hidden

The input is converted to tokens (via adapters), then 70% are randomly removed. The model only gets to see the remaining 30%. Its job is to figure out what the hidden 70% looked like — forcing it to learn the underlying structure.

Step 3

Shared ViT Encoder — 8 Blocks, ALL trainable

Processes only visible tokens. Each block applies self-attention (see Slide 2) to build rich representations. During pretraining, every weight can update.

Step 4

Decoder — reconstructs masked tokens
4 lightweight blocks. Inserts learnable mask tokens, predicts originals. Discarded after training.

Loss = MSE between predicted and actual masked tokens → backpropagate → update all weights

Why self-supervised pretraining?

Labeled wireless data is expensive to collect. But unlabeled radio recordings are abundant. By training the model to reconstruct masked signals, it learns general patterns:

- What WiFi bursts look like vs. Bluetooth hops
- How multipath propagation affects channel responses
- Temporal and spatial structure in IQ waveforms

This general knowledge transfers to many downstream tasks.

Training details

- 800 epochs, 40-epoch linear warmup
- Adam optimizer, learning rate = 0.001
- Cosine LR schedule (gradually decreases LR)
- Alternates: 1 batch of IQ, then 1 batch of spectrograms
- Both modalities share the same encoder weights

What pretraining produces:
A checkpoint file (wavesfm-v1p0.pth) containing encoder weights that capture general wireless knowledge. This is then reused in Phase 2 (fine-tuning) for specific tasks.

The decoder is DISCARDED after pretraining.
Only the encoder moves on to Phase 2. The decoder was just a training tool.

Inside a Transformer Block

Each of the 8 encoder blocks performs these 3 operations on every token

Input Tokens
(list of 256-d vectors)

→

Step 1: Self-Attention
Each token looks at ALL other tokens to decide which ones are relevant

↓

How Self-Attention works (step by step):

1. Each token is transformed into 3 vectors:
   Query (Q) — "what am I looking for?"
   Key (K) — "what do I contain?"
   Value (V) — "what information do I carry?"

2. Compute attention scores: Q × K (dot product)
   → *tells how relevant each other token is*

3. Weighted sum of V using those scores
   → *each token gets updated based on relevant context*

Example: WiFi sensing

Imagine a CSI image split into 196 patches (tokens).

A patch showing signal fading in one corner can "attend" to patches showing strong signal elsewhere. This lets the model understand the spatial pattern of signal propagation across the entire image at once.

Multi-head attention (8 heads) means 8 different "views" of relevance are computed in parallel, each capturing different types of relationships.

Step 2: Feed-Forward Network
Each token is processed independently through 2 linear layers with a nonlinearity in between. Expands to 1024-d, then back to 256-d.

Step 3: Layer Normalization
Normalizes each token to have stable mean and variance.
Prevents values from exploding or vanishing across blocks.

Residual connections: the output of each step is ADDED to its input (output = input + step_result).
This means if a step learns nothing useful, the original information still passes through unchanged.
Stacking 8 blocks means 8 rounds of attention → FF → norm, progressively refining the representation.

Phase 2: Fine-Tuning on Downstream Tasks

Reusing the pretrained encoder for specific wireless tasks

IQ Signals
Raw waveforms (amplitude + phase)

Image-like Wireless Data
Spectrograms, CSI, channel maps

IQ Adapter
Segments → 256-d tokens + position + antenna info
Always trainable (adapts to new signal formats)

Adapters convert different data shapes into the SAME token format (256-d vectors)

Vision Adapter
16×16 patches → 256-d tokens + spatial position
Always trainable (adapts to new image formats)

Modality Conditioning — aligns both token types to same numerical scale

Shared ViT Encoder — loads pretrained weights from wavesfm-v1p0.pth

Encoder already knows useful patterns from pretraining

Block 1	Block 2	Block 3	Block 4	Block 5	Block 6	Block 7	Block 8
Attn→FF→Norm	Attn→FF→Norm	Attn→FF→Norm	Attn→FF→Norm	Attn→FF→Norm	Attn→FF→Norm	Attn→FF→Norm	Attn→FF→Norm

Blocks 1–6: FROZEN (pretrained, locked)

Blocks 7–8: mode-dependent (see Slide 4)

Classification Head
"which category?" → class label

Regression Head
"what location?" → x,y meters

Beam Head
"which beam?" → 1 of 64

Three Fine-Tuning Modes

How much of the pretrained encoder is allowed to change for a new task

LP — Linear Probe

1 2 3 4 5 6 7 8

ALL 8 blocks FROZEN

Only the task head learns

What this tests:
Are the pretrained features already good enough on their own? If LP works well, the pretraining was effective.

FT2 — Fine-Tune 2 Blocks

1 2 3 4 5 6 7 8

1–6: FROZEN 7–8: TRAIN

Head + blocks 7–8 learn (~1.65M params)

What this tests:
Can the last 2 blocks adapt their higher-level features while keeping early general features intact?

LoRA — Low-Rank Adaptation

1 2 3 4 5 6 7 8

ALL 8 FROZEN + LoRA adapters inside

Head + LoRA adapters learn (~0.3M params)

What this tests:
Can tiny adapter matrices achieve similar adaptation with 5× fewer parameters? (See Slide 5 for details)

Trainable parameters: LP ≈ head only | FT2 ≈ 1.65M (head + 2 blocks) | LoRA ≈ 0.3M (head + adapters) | Full model = 6.32M

What is a "task head"?

A small output layer — usually just a single linear transformation — attached on top of the encoder.
Each task gets its own head. The head is ALWAYS trained from scratch, even when the encoder is frozen.
It translates the encoder's general 256-d features into a task-specific answer: a class label, a position, or a beam index.

Key insight: If LP (zero encoder adaptation) still performs well, it means the pretrained features are genuinely useful.
If FT2 or LoRA improve further, it means task-specific adaptation helps but the foundation is already strong.

LoRA Deep Dive: How Low-Rank Adaptation Works

What "projection" and "rank" mean, and why LoRA is so parameter-efficient

The problem: full weight matrices are huge

Each attention block has Query (Q) and Value (V) projections.
Each is a 256 × 256 matrix = 65,536 parameters.
Updating all of them is expensive and risks forgetting
what the model learned during pretraining.

LoRA solution: add a small low-rank correction

Keep the original 256×256 matrix FROZEN.
Add two tiny matrices alongside it that learn a correction.
The correction has low "rank" — it squeezes through a
small bottleneck, keeping the number of parameters tiny.

Step-by-step: what happens inside one LoRA adapter

Input Vector
256 numbers
(one token)

→

Matrix A
256 → 32
"Project DOWN"
Compress to 32 dims

→

Bottleneck
32 numbers
This is the "rank"
(the narrow point)

→

Matrix B
32 → 256
"Project BACK UP"
Expand to 256 dims

→

Small Correction
256 numbers
Added to the frozen
original output

Final output = (Frozen original matrix × input) + (α / √r) × B × A × input

The original weights are NEVER modified. The A×B product is a learned correction added on top.

Parameter count per Q or V projection:

Full matrix: 256 × 256 = 65,536 parameters
LoRA (A+B): 256×32 + 32×256 = 16,384 parameters
That is 4× fewer parameters per projection.

Why "low-rank" works:

The assumption is that task-specific adaptation
lives in a much smaller subspace than the full
256 dimensions. Rank 32 is enough to capture
what needs to change for a new task.